

Labo 01 — Questions

Répondez sans relire la théorie. Une réponse d'une à cinq phrases suffit ; ce qui compte est le raisonnement, pas le vocabulaire.

Question 1 [Compréhension]

Un collègue affirme : « Un conteneur, c'est une petite machine virtuelle avec un Linux minimal dedans. » Dites ce qui est faux dans cette phrase, et expliquez pourquoi cette confusion a des conséquences concrètes sur le temps de démarrage et l'occupation disque.

Question 2 [Analyse]

L'image `postgres:16-alpine` pèse environ 250 Mo et contient une arborescence Linux complète (`/bin`, `/etc`, `/usr`...). Pourtant on dit qu'« il n'y a pas d'OS dans un conteneur ». Les deux affirmations sont vraies : expliquez ce qui se trouve réellement dans cette image et ce qui, dans un système d'exploitation, en est **absent**.

Question 3 [Analyse]

Vous lancez deux conteneurs à partir de la même image `nginx:alpine`. Sur l'hôte, `ps aux | grep nginx` montre deux jeux de processus. Pourtant, depuis l'intérieur du premier conteneur, `ps` ne montre que ses propres processus. Quel mécanisme du noyau est responsable, et pourquoi ce n'est **pas** de la sécurité au sens strict ?

Question 4 [Diagnostic]

Un collègue qui utilise Docker vous montre ceci :

```
Client: Docker Engine - Community
Version: 29.7.2
Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
Is the docker daemon running?
```

Expliquez ce qui s'est passé chez lui (deux causes probables), pourquoi il est inutile de modifier sa commande — et pourquoi ce message ne peut **pas** vous arriver avec votre Podman rootless sous WSL.

Question 5 [Compréhension]

« Une image ne tourne pas. » Justifiez cette phrase, puis expliquez ce que le moteur ajoute concrètement à l'image au moment où vous lancez `podman run`.

Question 6 [Analyse]

Vous démarrez un conteneur PostgreSQL, vous y créez une base et des tables, puis vous faites `podman rm` sur ce conteneur. Vous relancez un nouveau conteneur à partir de la **même image**. Vos données sont-elles là ? Justifiez en vous appuyant sur la structure image / couche d'écriture — et dites si l'image a été modifiée par votre travail.

Question 7 [Analyse]

Vous lancez dix conteneurs de l'image `eclipse-temurin:21-jre-alpine` (environ 210 Mo). Quel espace disque supplémentaire cela consomme-t-il, approximativement ? Expliquez le mécanisme qui rend cette réponse possible.

Question 8 [Diagnostic]

Dans un conteneur Podman rootless, `id` affiche `uid=0(root)`. Sur l'hôte WSL, `podman top <conteneur> user,huser` affiche `root` dans la colonne USER et `1000` dans la colonne HUSER. Expliquez ce que signifie cette double identité, quel namespace la produit, et ce que peut réellement faire ce « root » s'il tente d'écrire dans `/etc/shadow` de l'hôte via un montage.

Question 9 [Compréhension]

Votre entreprise interdit d'ajouter des utilisateurs au groupe `docker` sur les serveurs de production, et exige de passer par `sudo` avec une trace d'audit. Quel est le raisonnement de sécurité derrière cette règle, et en quoi Podman rootless rend-il la question sans objet ?

Question 10 [Analyse]

Traduisez en forme longue (`podman <objet> <action>`) les commandes suivantes, et dites pour chacune sur quel **type d'objet** elle agit :

```
podman ps -a
podman images
podman rmi nginx:alpine
podman rm web
```

Pourquoi `podman ps` et `podman images` ne suivent-ils pas la même logique de nommage ?

Question 11 [Diagnostic]

Depuis votre terminal Ubuntu sous WSL, `podman run --rm alpine uname -r` affiche `6.6.87.2-microsoft-standard-WSL2`. Un collègue sur un serveur Ubuntu natif obtient `6.8.0-45-generic` avec la même commande. Expliquez d'où vient chacune de ces valeurs, et ce que cela implique pour l'affirmation « les conteneurs sont légers » sur un poste Windows.

Question 12 [Analyse]

Une application conteneurisée mal codée part en boucle infinie et consomme toute la RAM disponible. Le namespace `pid` l'empêche-t-il de nuire aux autres conteneurs ? Quel est le mécanisme correct à mobiliser, et que se passe-t-il si personne ne l'a configuré ?

Question 13 [Compréhension]

En entreprise, l'image du back Spring Boot construite par la CI est promue de l'intégration vers la recette puis la production **sans être reconstruite**, et les serveurs de recette tournent sous Docker alors que la production tourne sous Podman. Quelles propriétés rendent cette pratique possible, et quel risque prend-on si l'on reconstruit l'image à chaque étape à partir du même code source ?

Question 14 [Analyse]

Un développeur ajoute `alias docker=podman` dans son `.bashrc` et affirme que « tout ce qui est écrit pour Docker marchera ». Donnez deux exemples où c'est vrai sans réserve, et deux situations où l'architecture différente de Podman (pas de daemon, rootless) change concrètement le comportement observé.